



**INSTITUTO
FEDERAL**

Espírito Santo

Campus
Santa Teresa

Aula 33 – Manipulação do DOM e Eventos

Interatividade com JavaScript

Prof. Matheus Jagi

27 de Maio de 2026

O que é DOM?

- DOM = Document Object Model
- Representação da página HTML em forma de objetos

Objetivo

- Permitir que JavaScript manipule a página

Podemos:

- alterar textos
- modificar estilos
- adicionar elementos
- responder eventos

0 navegador transforma HTML em:

- árvore de elementos

Exemplo

- `html`
 - `body`
 - `div`
 - `h1`
 - `button`
- JavaScript acessa essa estrutura

Estrutura inicial do projeto

- Criar uma pasta para o projeto

Exemplo

```
meu-projeto/  
├──  
├── index.html  
├── style.css  
└── script.js
```

Preparando o ambiente II

Função de cada arquivo

- `index.html` → estrutura da página
- `style.css` → estilos
- `script.js` → lógica JavaScript

Criando o arquivo HTML I

```
<!DOCTYPE html>
<html lang="pt-BR">

<head>
  <meta charset="UTF-8">
  <title>DOM e Eventos</title>

  <link rel="stylesheet"
        href="style.css">
</head>

<body>

  <h1 id="titulo">
    Olá Mundo
  </h1>

  <button id="btn">
    Clique aqui
  </button>

  <script src="script.js"></script>

</body>
</html>
```

Criando o arquivo HTML II

Importante

- `script.js` deve ser importado no final do body

Motivo

- Garantir que HTML já foi carregado

Arquivo:

● script.js

```
const titulo =  
document.querySelector("#titulo");  
  
const btn =  
document.querySelector("#btn");  
  
btn.addEventListener(  
  "click",  
  function() {  
    titulo.textContent =  
      "Texto alterado!";  
  }  
);
```


Criando o arquivo JavaScript II

O que acontece?

- JavaScript captura clique do botão
- Texto do h1 é alterado

Utilizando o Console I

Ferramenta importante

- Console do navegador

Abrir

- F12
- Aba Console

Exemplo

```
console.log("Teste");
```

Objetivo

- Verificar erros
- Exibir informações

Arquivo JS não encontrado

- Nome incorreto
- Caminho errado

Elemento não encontrado

- ID errado
- Script executando antes do HTML carregar

Solução

- Conferir nomes e estrutura

Passo 1

- Criar HTML

Passo 2

- Adicionar IDs/classes

Passo 3

- Selecionar elementos no JS

Passo 4

- Criar eventos

Passo 5

- Testar no navegador

Selecionando elementos I

querySelector()

```
const titulo =  
document.querySelector("h1");
```

Seleciona

- primeiro elemento encontrado

Pode usar

- tag
- classe
- id

Selecionando por ID I

```
<h1 id="titulo">  
  Olá Mundo  
</h1>
```

```
const elemento =  
document.querySelector("#titulo");
```

Símbolo

- representa ID

Selecionando por classe I

```
<p class="texto">  
  Conteúdo  
</p>
```

```
const texto =  
document.querySelector(".texto");
```

Símbolo .

- representa classe

Alterando conteúdo I

textContent

```
titulo.textContent =  
"Novo título";
```

innerHTML

```
titulo.innerHTML =  
"<strong>Texto</strong>";
```

Diferença

- `textContent` → texto puro
- `innerHTML` → interpreta HTML

Alterando imagem

```
imagem.src = "foto.jpg";
```

Alterando links

```
link.href =  
"https://google.com";
```

classList

```
caixa.classList.add("ativo");  
  
caixa.classList.remove("ativo");  
  
caixa.classList.toggle("ativo");
```

Muito utilizado para

- menus
- dark mode
- animações

Introdução aos Eventos I

O que são eventos?

- Ações realizadas pelo usuário

Exemplos

- click
- input
- submit
- mouseover

JavaScript pode “escutar” eventos

addEventListener I

Estrutura básica

```
botao.addEventListener(  
  "click",  
  function() {  
    alert("Clicou!");  
  }  
);
```

Explicação

- click → evento
- função → ação executada

Evento de click I

HTML

```
<button id="btn">  
  Clique  
</button>
```

JavaScript

```
const btn =  
document.querySelector("#btn");  
  
btn.addEventListener(  
  "click",  
  function() {  
    console.log("Botão clicado");  
  }  
);
```

Capturando digitação

```
campo.addEventListener(  
  "input",  
  function() {  
    console.log(campo.value);  
  }  
);
```

Muito usado em:

- buscas
- filtros
- formulários

Formulários

```
form.addEventListener(  
  "submit",  
  function(event) {  
    event.preventDefault();  
    console.log("Enviado");  
  }  
);
```

preventDefault()

- impede recarregamento da página

Fluxo básico de interação I

Usuário interage



Evento acontece



JavaScript captura



Interface é atualizada

Exemplo integrado I

HTML

```
<input id="nome">  
  
<button id="btn">  
  Salvar  
</button>  
  
<p id="resultado"></p>
```

JavaScript

```
const nome =  
document.querySelector("#nome");  
  
const btn =  
document.querySelector("#btn");  
  
const resultado =  
document.querySelector("#resultado");  
  
btn.addEventListener(  
  "click",  
  function() {  
    resultado.textContent =  
      nome.value;  
  }  
);
```

Evitar JavaScript inline

- usar `addEventListener`

Organizar seletores

- nomes claros

Separar responsabilidades

- HTML → estrutura
- CSS → estilo
- JS → comportamento